



RS6x_7x mmWave Radar Point Cloud Processing Reference Manual

Number of Pages: 33

Document Version: 1.2

Release Date: 2026-9-4

Revision History

Version	Revision Description	Date
1.2	Revised the description of the presence point cloud detection methods in Section 3.2; Revised the description of the presence point cloud detection thresholds in Section 4.3.2; Added the presence point cloud execution time for each detection method; Added a Q & A item;	2026-09-03
1.1	Modify section 2.3 format error; Updated the radar coordinate system definition in Figure 2-11;	2026-08-13
1.0	Initial release;	2026-06-13

Table of Contents

Revision History	ii
Table of Contents	iii
List of Figures	v
List of Tables	vi
1 Purpose and Scope	1
1.1 Component Overview	1
1.2 Code and Tools	1
1.2.1 Source Code Path	1
1.3 Related Documents	2
2 Moving Point Cloud Digital Signal Processing	3
2.1 CFAR Target List	4
2.1.1 Hardware CFAR Target List	4
2.1.2 Software CFAR Target List	4
2.2 Noise Floor Estimation	5
2.3 Point Cloud Filtering	5
2.3.1 Low-Speed Filtering	6
2.3.2 Range-Doppler Filtering	8
2.3.3 SNR Filtering	8
2.3.4 Point Cloud Filtering Algorithm	8
2.4 Peak Grouping Module	9
2.5 Angle Measurement Module	10
2.5.1 Basic Principles of Angle Measurement	10
2.5.2 RS6x_7x Antenna Arrangement	11
2.5.3 RS6x_7x Coordinate System Definition	13
2.5.3.1 Radar Coordinate System	13
2.5.3.2 User Coordinate System	14
2.5.4 RS6x_7x Angle Calibration	15
2.5.5 RS6x_7x Angle Measurement Interface	16
3 Presence Point Cloud Digital Signal Processing	18
3.1 Detection Method Selection	19
3.2 Angle Measurement	19
4 Configuring the Point Cloud Processing Subsystem	20
4.1 Baseband Processing Configuration	20
4.1.1 Static Clutter Removal Method	20
4.1.2 Static Clutter Convergence Time	21
4.1.3 Receive-Antenna Gain and Transmit-Antenna Attenuation	21
4.1.4 IF Filter Bandwidth	21
4.2 Moving Point Cloud Function Configuration	21
4.2.1 Maximum CFAR Target List Size Configuration	21
4.2.2 CFAR Threshold	21
4.2.3 Peak Grouping Configuration	21
4.2.4 Point Cloud Filtering Algorithm Configuration	21
4.2.5 Static Target Removal	21
4.2.6 Low-Speed Filtering	22
4.3 Presence Point Cloud Function Configuration	22
4.3.1 Maximum Range Supported by the Presence Point Cloud	22
4.3.2 Presence Point Cloud Detection Threshold	22
4.3.3 Number of Presence Point Cloud Intervals (log2)	22
4.3.4 Presence Point Cloud Doppler FFT Length (log2)	23
4.3.5 Presence Point Cloud Range Downsampling Ratio	23
4.3.6 Presence Point Cloud Interval Downsampling Ratio	23

4.4 Global Configuration	23
4.4.1 Chip Mounting Orientation	23
4.4.2 Adaptive Gain	23
4.4.3 Point Cloud Range	24
4.4.4 Point Cloud Velocity Range	24
5 Execution Time and Memory Usage	25
6 [Q & A]	26

List of Figures

Figure 1-1	Overall Radar Development Framework	1
Figure 2-1	Digital Signal Processing Data Flow for the Point Cloud Reference Design	3
Figure 2-2	Example of Software CFAR Continuation After the Hardware CFAR List Becomes Full	5
Figure 2-3	Low-Speed Filtering Flowchart	7
Figure 2-4	SNR Filtering	8
Figure 2-5	Point Cloud Filtering Results	9
Figure 2-6	Peak Grouping Illustration	9
Figure 2-7	Peak Grouping Illustration (Boundary)	10
Figure 2-8	Relationship Between Phase Difference and Angle of Arrival	10
Figure 2-9	RS6130/RS6240/RS6241 Antenna Arrangement	12
Figure 2-10	MISO Virtual Array of the RS6240 Antennas	12
Figure 2-11	Radar Coordinate System Definition	13
Figure 2-12	Positive Directions of the Radar Coordinate System	14
Figure 2-13	Development Board in the Side-Mounted Vertical Orientation	14
Figure 2-14	Positive Directions of the User Coordinate System	15
Figure 2-15	Phase Calibration Calculation Method	16
Figure 2-16	Angle Measurement Flow	16
Figure 3-1	Presence Point Cloud DataCube Processing Flow	18
Figure 4-1	Radar Callback Scheduling Sequence When Configured as MMW_CLUTTER_REMOVAL_DC	20
Figure 5-1	SRAM Usage of the Point Cloud Subsystem	25

List of Tables

Table 5-1	Execution Time per Call of Key Functions	25
Table 5-2	Presence Point Cloud Execution Time per Frame	25

1 Purpose and Scope

This document describes in detail the point cloud reference-design project implemented using the Possumic mmWave radar software development kit (SDK) components. After reading this document together with the embedded code, developers can understand how to use the APIs provided by the SDK to calculate point clouds from the DataCube or CFAR target list.

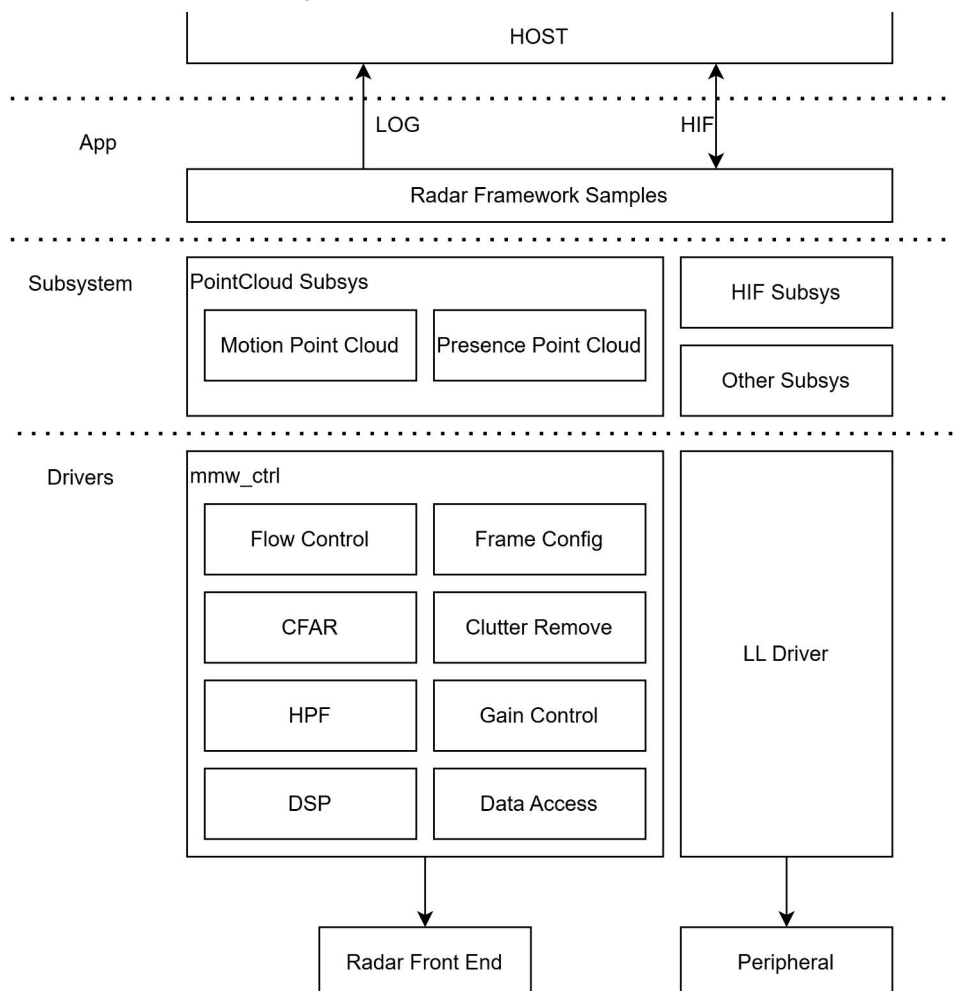


Figure 1-1 Overall Radar Development Framework

The content described in this document is located in the PointCloud Subsys shown in Figure 1-1 Overall Radar Development Framework.

1.1 Component Overview

The point cloud development example described in this document supports all chips.

1.2 Code and Tools

1.2.1 Source Code Path

The source code described in this document is located at:

[/Software Kit/02 SDK/psdf sdk/subsys/mmw/mmw_algorithm](#) and
[/Software Kit/02 SDK/psdf sdk/subsys/mmw/mmw_application](#)

1.3 Related Documents

For more information about using PointCloud Subsys at the application layer and evaluating point cloud performance, refer to the ['RS6x 7x mmWave Radar Sample Reference Manual.pdf'](#).

For more information about the driver-layer radar modules and SDK, refer to the ['RS6x 7x mmWave Radar Reference Manual.pdf'](#) and the ['RS6x 7x SDK Reference Manual.pdf'](#).

For information about using the hardware accelerators in PointCloud Subsys, refer to the ['RS6x 7x DSP Interface Description Manual.pdf'](#).

2 Moving Point Cloud Digital Signal Processing

In the point cloud digital signal processing module, developers read data from the DataCube and CFAR target list and calculate the polar-coordinate information of the point cloud through target filtering (1st pass and 2nd pass), calibration, angle estimation, and other steps.

The point cloud reference design provides a standard, concise, open-source, and typical digital signal processing flow to demonstrate how developers can complete point cloud calculations using basic APIs. This flow is referred to as the point cloud processing subsystem. After becoming familiar with this standard module, developers can add their own code to quickly build point cloud applications that meet their requirements.

The point cloud digital signal processing module consists of the following:

- Noise Estimation module;
- CFAR target list acquisition (CFAR);
- Point cloud filtering modules (1st Pass and 2nd Pass);
- Peak Grouping module (PeakGrouping);
- Angle Measurement Module (Direction Of Arrival(DOA))
- Coordinate Calculation module;

These six modules use data interfaces in MMW-CTRL and the radar algorithm library provided by the SDK. For the digital signal processing data-flow diagram, refer to Figure 2-1.

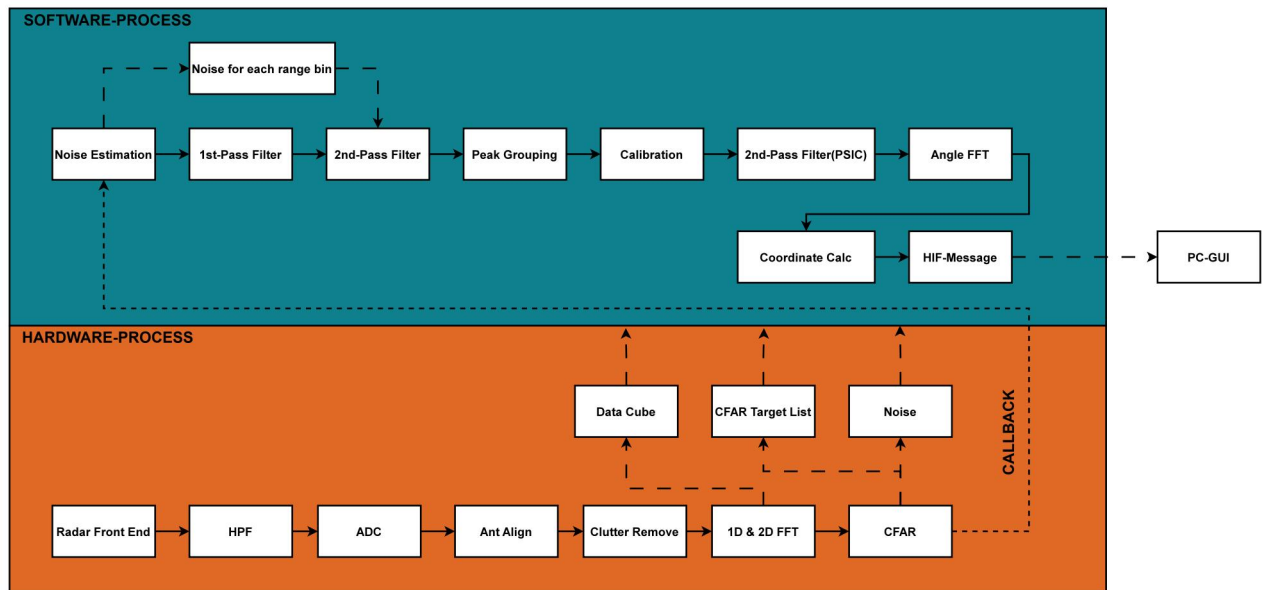


Figure 2-1 Digital Signal Processing Data Flow for the Point Cloud Reference Design

The important code structure of the point cloud processing subsystem is shown below.

```

/02_SDK/psdf_sdk/project/mmwave/
/* Point cloud processing subsystem */
<home>/subsys/
|__ mmw
    |__ mmw_algorithm
        |__ mmw_alg_pointcloud.c          /* Standalone API functions */
    |__ mmw_application
        |__ mmw_app_pointcloud_config.h    /* Point cloud subsystem configuration */
        |__ mmw_app_pointcloud.c          /* Point cloud application flow */
        |__ mmw_app_micro_pointcloud.c     /* Presence point cloud source code */

```

2.1 CFAR Target List

The CFAR target list consists of the target list calculated by hardware and the supplementary target list calculated by software. The point cloud processing module calls `mmw_cfar_detection_process()` to obtain the merged CFAR target list.

2.1.1 Hardware CFAR Target List

The hardware CFAR target list is obtained by calling `mmw_cfar_result()`. Hardware CFAR stores targets from far to near and supports a maximum target-list length of 512 points. If more than 512 targets are detected, newly detected targets are continuously overwritten. Before calling `mmw_cfar_result()`, do not call `mmw_motion_cube_access_open()`, because doing so causes the CFAR target list to be read incorrectly.

For more information about hardware CFAR, refer to the ['RS6x 7x mmWave Radar Reference Manual.pdf'](#).

During hardware DataCube calculation, `fft_shift` has already been performed, moving zero-velocity targets to the center of the Doppler dimension. For a Doppler FFT length of 32, hardware CFAR targets with a Doppler index of 16 indicate stationary targets. Targets at Doppler indices 0 to 15 have negative velocity (moving toward the radar), while targets at indices 17 to 31 have positive velocity (moving away from the radar). The actual radial velocity of a target is calculated as follows:

$$[\text{Doppler index} - (\text{Doppler FFT length} / 2)] * \text{velocity resolution}$$

2.1.2 Software CFAR Target List

When the hardware CFAR queue is full, the point cloud processing subsystem executes software CFAR from range zero to the point where the hardware CFAR list was interrupted. Software CFAR uses only Tx0-Rx0 data, calculates the SNR against the noise floor at the corresponding range bin, and compares it with the SNR threshold to obtain point cloud information.

Three different conditions may occur. For each condition, the range of the final CFAR list is shown in Figure 2-2.

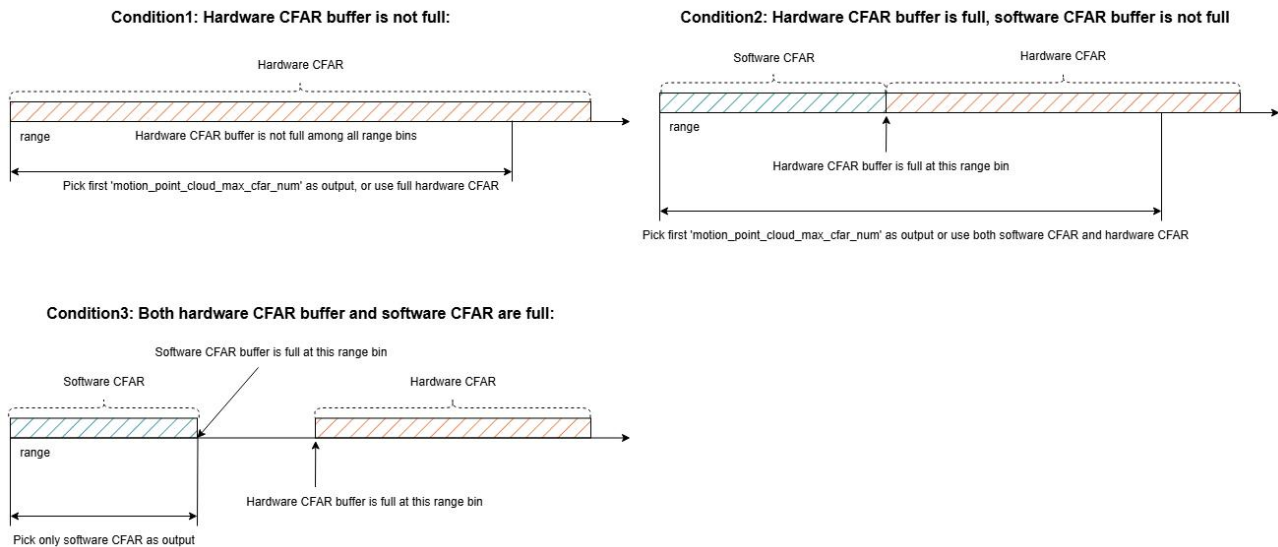


Figure 2-2 Example of Software CFAR Continuation After the Hardware CFAR List Becomes Full

2.2 Noise Floor Estimation

Because the SNR provided by the CFAR result has low precision, the SNR can be re-estimated in software. Noise floor estimation provides an accurate noise floor for SNR filtering. The noise estimation algorithm is provided by the radar algorithm library. It calculates the noise floor information of the current frame in each frame and outputs a smoothed noise floor.

The interfaces provided by the function are as follows:

- *mmw_psic_noise_coverage_flag*
- After *mmw_psic_lib_init()* is called, noise-floor estimation enters a fast-convergence state. The application must call *mmw_psic_noise_coverage_flag()* to determine whether fast convergence has completed. Before fast convergence completes, it is recommended to estimate the noise floor in every frame and, where possible, ensure that no moving targets are present in front of the radar.
- *mmw_psic_noise_update*
- Call this function to use the current-frame DataCube data to update the noise floor at every range index. The noise floor at each range index is updated independently through a low-pass filter to produce a smooth output. The radar field of view must not contain moving targets during the update, because they affect estimation accuracy. Whether a valid point cloud was present in the last frame can be used as the condition for updating the noise floor. With this method, an excessively low CFAR threshold that causes continuous false detections may prevent the noise floor from being updated normally.
- *mmw_psic_noise_get_f32*
- The radar algorithm library provides an interface for obtaining the noise floor in W. An interface with the f32 suffix reads the data as single-precision floating-point values. The obtained noise floor is the average noise-floor power of all receive channels at the specified range index. Because each call introduces processing overhead, avoid repeatedly calling this function to obtain the noise floor at the same range index.

2.3 Point Cloud Filtering

The targets output in the CFAR list include some false targets. High-performance applications must filter these false targets and reduce the total point cloud output to reduce subsequent processing load and improve point cloud quality. This process is referred to as point cloud filtering. It consists of four independent steps:

1. Low-speed filtering;
2. Filter targets in the CFAR target list that are too near or too far based on range and Doppler information;
3. Filter targets based on accurate SNR values;
4. Filter targets using the point cloud filtering interface provided by the radar algorithm library.

2.3.1 Low-Speed Filtering

Low-speed filtering performs Doppler interpolation for targets at the zero-velocity Doppler index to estimate their precise Doppler indices. A target is filtered when its interpolated Doppler index is below the configured threshold. This is used when targets at the zero-velocity Doppler index must be retained while targets that are completely stationary relative to the radar must be filtered from the environment. The algorithm is implemented by *mmw_process_low_speed_filter()*, which returns an array with the same length as the current CFAR result and indicates whether each corresponding CFAR point is filtered. The algorithm flow is shown in Figure 2-3:

1. Traverse all CFAR results;
2. Determine whether the current CFAR point is at zero velocity. Doppler interpolation is performed only for zero-velocity targets;
3. Determine whether the current target is a voltage peak within two velocity cells on each side. If it is not a peak, interpolation is not performed and the interpolation result is set to 1 or -1;
4. If Conditions 2 and 3 are met, perform Doppler interpolation. When the interpolation result is below the specified threshold, filter the current target and the targets within two velocity cells on each side. The Doppler interpolation interface is *mmw_zeros_vel_interpolation()*, and its return value is the interpolated offset relative to the zero-velocity index;
5. End after all CFAR results have been traversed.

Note that the interpolation algorithm is intended for FFT processing with a Hann window. If a rectangular window is configured, interpolation error is large and affects the performance of the low-speed filtering algorithm.

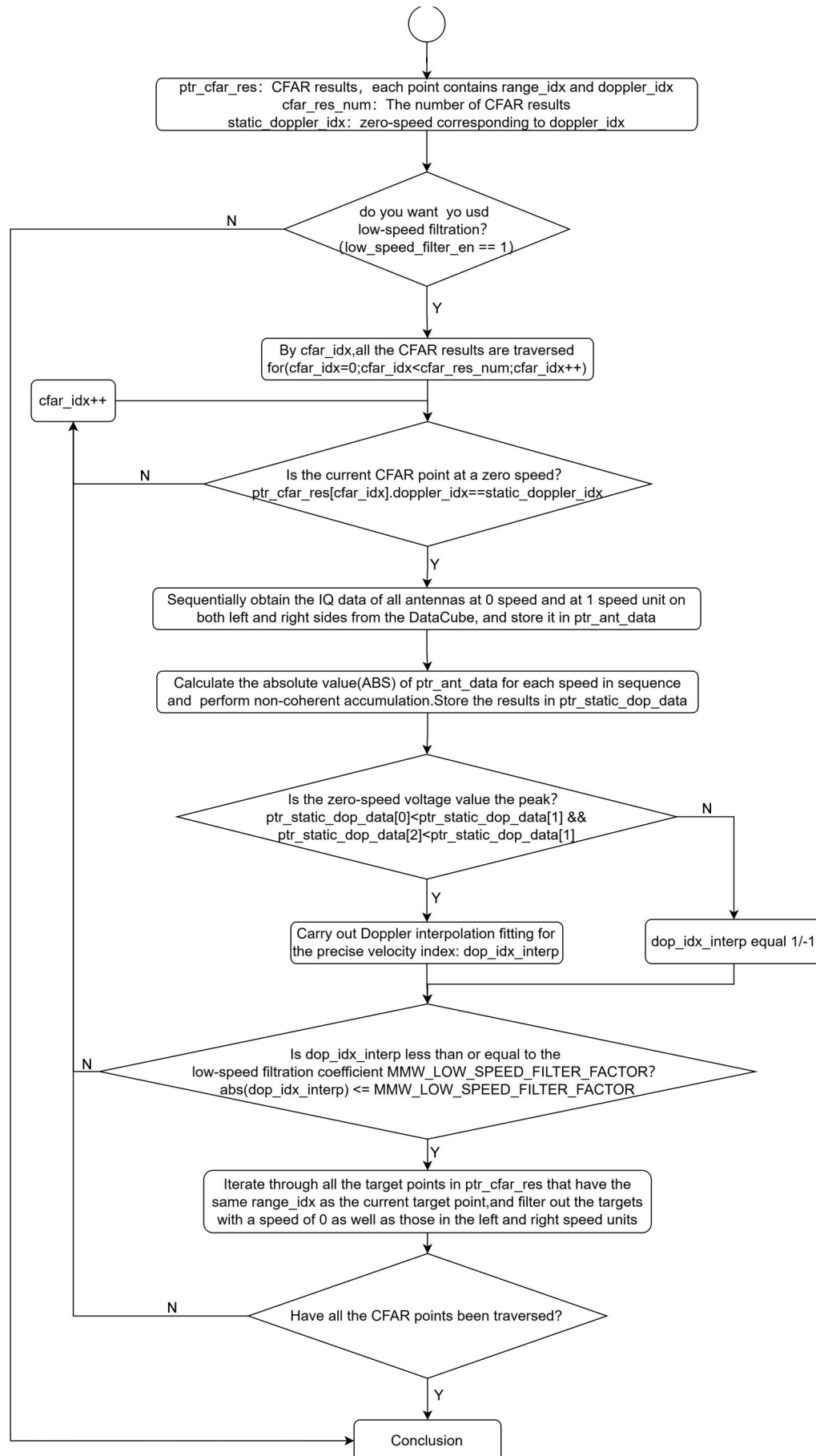


Figure 2-3 Low-Speed Filtering Flowchart

2.3.2 Range-Doppler Filtering

Due to hardware-processing characteristics, signals at very short ranges (typically range indices 0 to 3) and in the final 20% of the range indices may be subject to interference. These ranges must be excluded during point cloud filtering.

In some applications, if targets with very low Doppler velocity are not of interest, such as targets approaching the radar very slowly or moving perpendicular to the radar line of sight, it is recommended to exclude targets within two velocity cells on each side of zero velocity. This reduces the possibility that leakage from stationary targets into these velocity cells is detected as targets.

The point cloud processing submodule also supports customized valid range-index and Doppler-index ranges. Only point clouds within these valid ranges are calculated, which can significantly reduce the computational load.

In the point cloud reference design, range-Doppler filtering is implemented by *mmw_process_1st_pass_filter()*. In the range dimension, targets outside the desired detection range and targets beyond 0.8 times the maximum range are filtered. In the velocity dimension, targets within one velocity cell on each side of zero velocity are filtered.

2.3.3 SNR Filtering

As described in the Hardware CFAR section, the SNR estimation of hardware CFAR has low precision and may select targets below the desired SNR threshold. During SNR filtering, the sum of the target power across all antennas is recalculated and compared against the threshold. The threshold equals the noise floor at the current range index obtained by *mmw_psic_noise_get_f32()*, multiplied by the number of antennas and then by the SNR margin.

The data sources and process flow are shown in Figure 2-4:

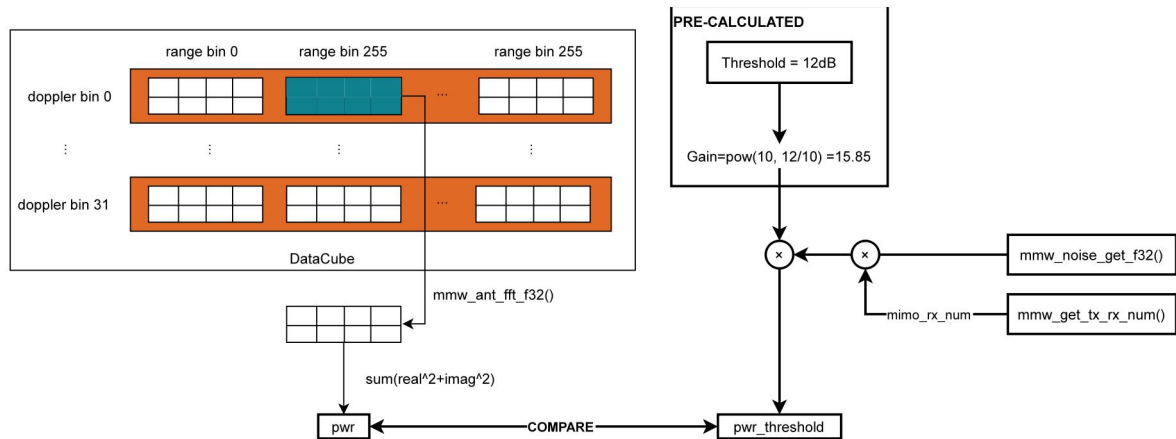


Figure 2-4 SNR Filtering

In the point cloud reference design, *mmw_process_2nd_pass_filter()* is used for SNR filtering. The SNR threshold is set to 12 dB, corresponding to a linear gain of 15.85.

2.3.4 Point Cloud Filtering Algorithm

The radar algorithm library provides a point cloud filtering algorithm that tests the characteristics of IQ data at specified range and Doppler indices to determine whether a target is a valid point.

Call *mmw_psic_2nd_pass_filter()* to execute the point cloud filtering algorithm. This function provides two filtering algorithms; developers may use one or both depending on the available processing resources. This function is effective only in 2T4R mode on the RS6240. On the RS6130, it does not filter the point cloud.

Figure 2-5 shows the results with *mmw_psic_2nd_pass_filter()* enabled (blue points) and disabled (red points). The test scene is a multi-frame accumulated point cloud from one person walking back and forth indoors. The x-y coordinates show point positions in meters. As shown, the number of false targets in a single frame is significantly reduced after *mmw_psic_2nd_pass_filter()* is enabled.

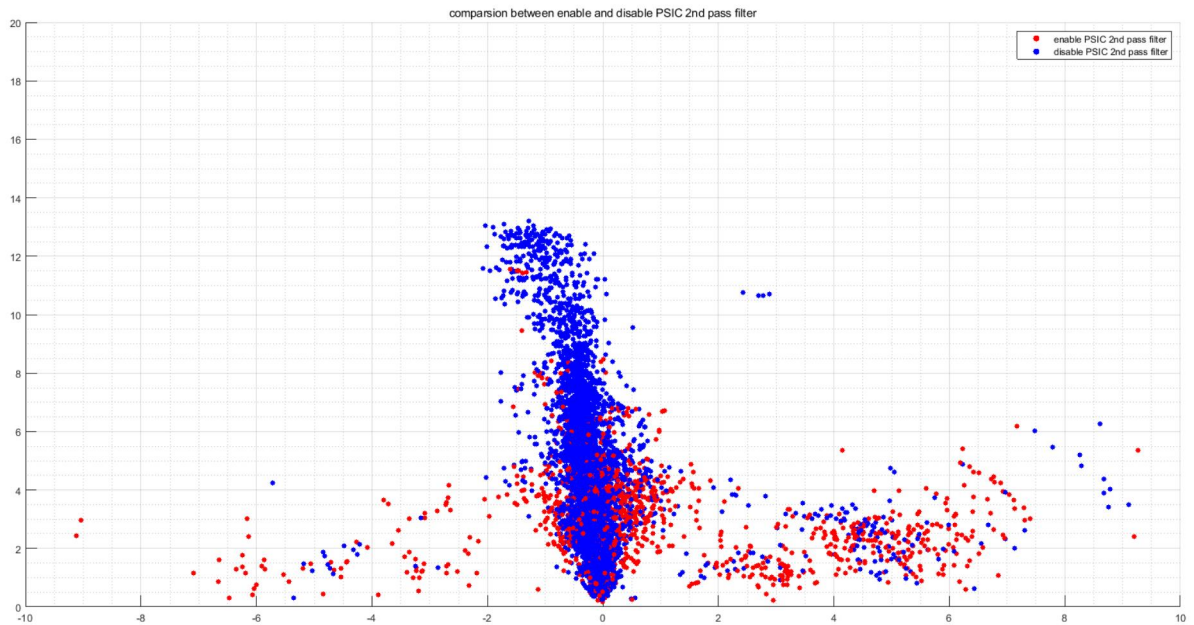


Figure 2-5 Point Cloud Filtering Results

2.4 Peak Grouping Module

Peak grouping can significantly reduce the number of output points and the load on subsequent processing. Its basic principle is to determine whether the point under test is a local power maximum. Depending on how the local power maximum is determined, the point cloud reference design provides three peak-grouping methods: PEAKGROUPING_DOPPLER, PEAKGROUPING_DOPPLER_RANGE, and PEAKGROUPING_DOPPLER_RECTANGLE. The differences among these methods are shown in Figure 2-6:

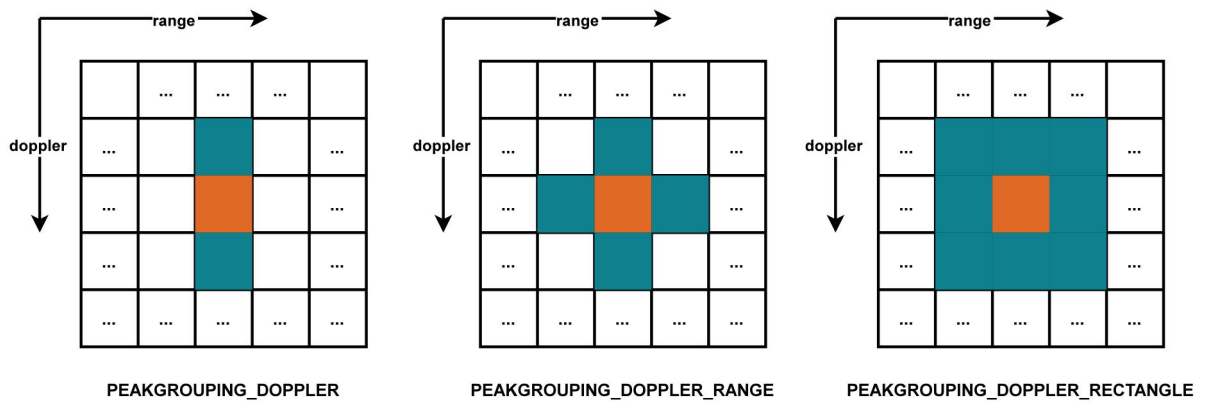


Figure 2-6 Peak Grouping Illustration

The orange block indicates the target location in the DataCube, and the green blocks indicate the power values of points in the neighboring area that must be checked.

When the range and velocity are at the range-FFT boundaries, the peak-grouping strategies differ. The following figure uses 256 range indices and 32 Doppler indices as an example:

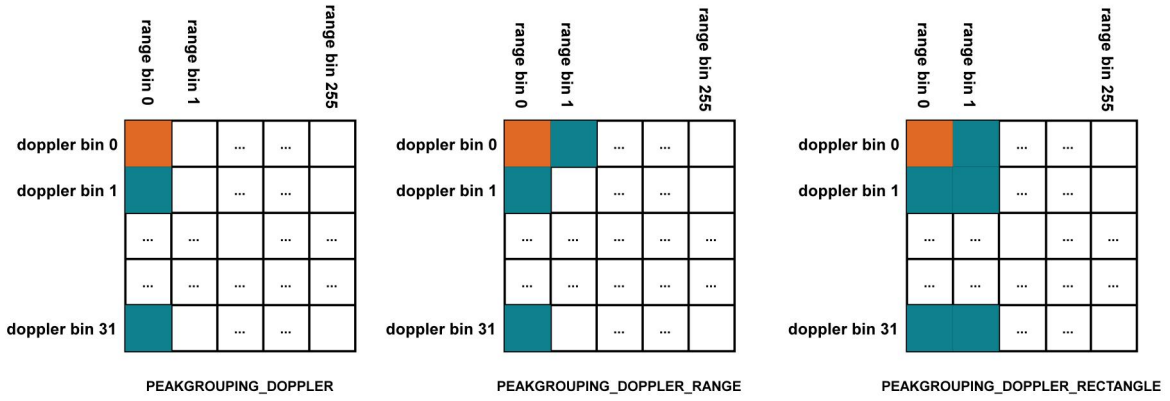


Figure 2-7 Peak Grouping Illustration (Boundary)

When the range index of the target under test is at a boundary, only valid neighboring range indices are used for peak determination. For range index 0, only the point at range index 1 is used. For range index 255, only the point at range index 254 is used.

When a Doppler index is at a boundary, the required indices wrap to the maximum or minimum Doppler index. If the Doppler index under test is k , its adjacent indices in the Doppler direction are: k_{left}, k_{right}

$$k_{left} = \begin{cases} k-1 & k-1 \geq 0 \\ k-1 + \text{dop_fft_len} & \text{else} \end{cases}$$

$$k_{right} = \begin{cases} k+1 & k+1 < \text{dop_fft_len} \\ k+1 - \text{dop_fft_len} & \text{else} \end{cases}$$

2.5 Angle Measurement Module

The angle measurement module calculates the phase difference between calibrated receive channels to obtain the sine of the corresponding direction angle. For a uniform array, angle FFT with interpolation is a computationally efficient approach and is used in the point cloud reference design.

2.5.1 Basic Principles of Angle Measurement

Angle measurement calculates the angle of arrival by determining the path-length difference of a point target across receive channels.

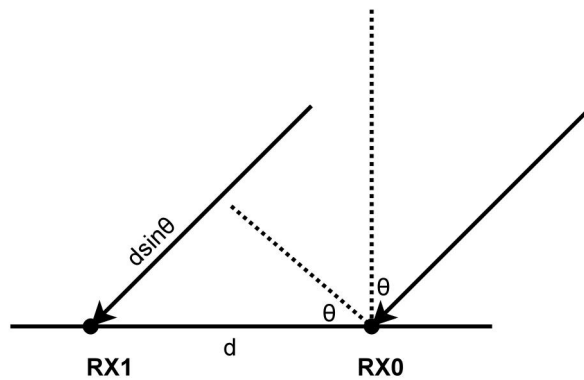


Figure 2-8 Relationship Between Phase Difference and Angle of Arrival

For a 1-transmit, 2-receive system, the preceding figure shows a top view of the radar. Rx0 and Rx1 are separated by d , and the target angle of arrival is θ . The positive direction of θ is defined as clockwise rotation from the radar boresight to the target. For a far-field radar target, the echoes can be regarded as parallel incident waves. Therefore, the phase difference between the two receive antennas depends on antenna spacing and angle of arrival θ . Let the f radar center frequency be f_c , $\lambda = c / f$ and the c corresponding wavelength be λ , where c is the speed of light, typically 3×10^8 m/s. The phase difference between the two receive antennas is therefore: ω

$$\omega = 2\pi d \sin \theta / \lambda$$

For a uniform linear array, the phase difference between every pair of adjacent antennas is identical. Under ideal conditions, this set of receive-array signals therefore has an identical phase progression: ω

$$Ae^{j\omega} [1 \quad e^{j\omega} \quad e^{j2\omega} \quad \dots \quad e^{j(N-1)\omega}] \quad \omega = 2\pi d \sin \theta / \lambda$$

An FFT can then be performed to calculate the phase difference ω and obtain the sine of the angle of arrival. The preceding system uses one transmit channel (Tx) and multiple receive channels (Rx) and is referred to as a (**M**ulti-**I**n-**S**ingle-**O**ut, **MISO**) system. A Multiple-Input (**M**ulti-**I**n-**M**ulti-**O**ut, **MIMO**) system can also be converted into an equivalent MISO system.

Taking a 1 transmit 4 receive MISO system as an example, in a certain frame channel receiving signal Rx0/Rx1/Rx2/Rx3, an example of the angle measurement process using 16-point angle FFT to measure the angle is explained.

1. Perform a 16-point angle FFT on [Rx0/Rx1/Rx2/Rx3] and calculate the squared magnitude of the FFT output. Assume that the peak is at index k, where indexing starts at 0;
2. Because the FFT length is small, parabolic interpolation must be used on the index to improve estimation accuracy. The parabolic interpolation method is:

$$k_{\text{interp}} = k + \frac{X_{k+1} - X_{k-1}}{4X_k - 2X_{k+1} - 2X_{k-1}}$$

In the $X_{k+1} X_k$ preceding X_{k-1} equation, the three terms correspond to the squared magnitudes of the FFT output at indices k+1, k, and k-1, respectively, and the final term is the interpolated k_{interp} index;

3. k_{interp} Apply fft_shift to the index to convert the domain to $[-\text{fft_len}/2, \text{fft_len}/2)$:

$$k_{\text{interp}} = \begin{cases} k_{\text{interp}} & k_{\text{interp}} < \text{fft_len}/2 \\ k_{\text{interp}} - \text{fft_len} & \text{else} \end{cases}$$

4. The resulting k_{interp} value $[-\pi, \pi)$ corresponds to a phase within the specified interval. This phase is the inter-antenna phase difference described at the beginning of this section, which varies with the angle of arrival and antenna ω spacing. The following equation can be used to obtain the sine of the angle of arrival:

$$\omega = 2\pi d \sin \theta / \lambda = k_{\text{interp}} \frac{2\pi}{\text{fft_len}} \Rightarrow \sin \theta = \frac{k_{\text{interp}}}{\text{fft_len}} \frac{\lambda}{d}$$

For RS6240/RS6130 series devices, the corresponding $\frac{\lambda}{d}$ value can be approximated as 2, giving:

$$\sin \theta = \frac{2k_{\text{interp}}}{\text{fft_len}}$$

If the chip has only two antennas in a given direction, [Rx0, Rx1], arranged as shown in Figure 2-8, the following phase measurement method can be used directly: $\sin \theta$

$$\begin{aligned} \phi &= \text{Angle}(Rx1 \bullet \text{conj}(Rx0)) = 2\pi \sin \theta \frac{d}{\lambda} \\ \Rightarrow \sin \theta &= \frac{\lambda \text{Angle}(Rx1 \bullet \text{conj}(Rx0))}{2\pi d} \end{aligned}$$

In the preceding equation, conj denotes complex conjugation and Angle denotes the phase angle of a complex number. For $z = a + bi$, $\text{Angle}(z) = \text{atan2}(b, a)$.

2.5.2 RS6x_7x Antenna Arrangement

RS624x and RS724x devices implement MIMO systems, whereas RS613x devices implement MISO systems. The angle measurement algorithm requires the correct antenna arrangement as input. Figure 2-9 shows the antenna numbering for RS6x_7x series devices.

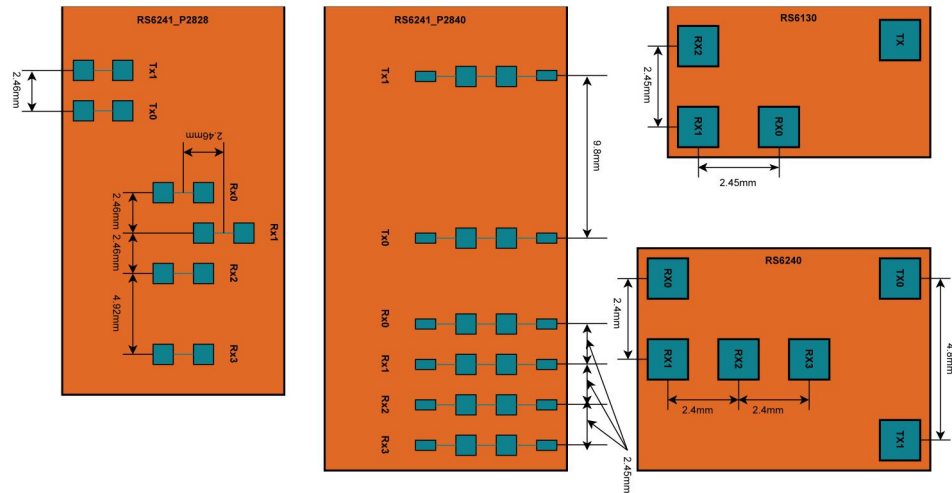


Figure 2-9 RS6130/RS6240/RS6241 Antenna Arrangement

The RS6130 is a MISO system and can therefore be used directly for azimuth and elevation measurement. The RS6240 and RS6241 are MIMO systems and require the MIMO antenna array to be virtualized as a MISO system. With the RS6240 orientation shown in Figure 2-9, the virtual array is as follows:

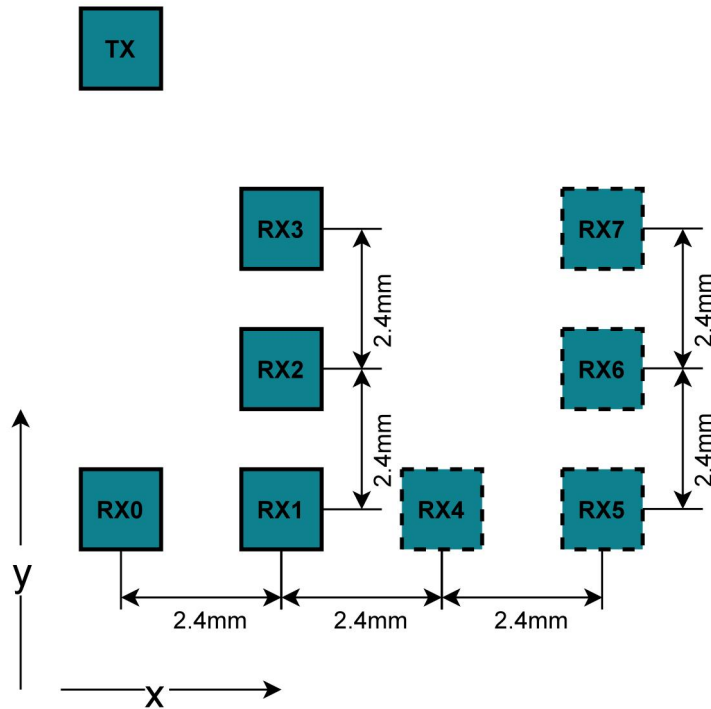


Figure 2-10 MISO Virtual Array of the RS6240 Antennas

Rx4 to Rx7 are the virtual receive channels obtained by aligning Tx1-through-Rx0 to Rx3 with Tx0. The spacing of each antenna pair Rx0/Rx4, Rx1/Rx5, Rx2/Rx6, and Rx3/Rx7 equals the spacing between Tx0 and Tx1.

The MIMO virtual-array principle is as follows: the coordinates of the array virtualized through Tx1 equal the coordinates of Rx0/Rx1/Rx2/Rx3 corresponding to Tx0, translated by the displacement vector from Tx0 to Tx1.

In the preceding example, the x and y directions of the antenna-array coordinate system are shown in the figure, with the origin at Rx0. The displacement vector from Tx0 to Tx1 is [4.8 mm, 0]. The coordinates of Rx0/Rx1/Rx2/Rx3 are [0, 0], [2.4 mm, 0], [2.4 mm, 2.4 mm], and [2.4 mm, 4.8 mm]. Therefore, the coordinates of the antennas virtualized by Tx1 are Rx0'=Rx4=[0+4.8 mm, 0+0], Rx1'=Rx5=[2.4+4.8 mm, 0+0], Rx2'=Rx6=[2.4+4.8 mm, 2.4 mm+0], and Rx3'=Rx7=[2.4+4.8 mm, 4.8 mm+0], producing the virtual-array arrangement shown above.

2.5.3 RS6x_7x Coordinate System Definition

The point cloud reference design uses two coordinate systems: the radar coordinate system and the user coordinate system. The radar first calculates target coordinates in the radar coordinate system and then converts them to the user coordinate system.

2.5.3.1 Radar Coordinate System

RS6x_7x devices support two radar coordinate-system definitions: vertical and horizontal mounting.

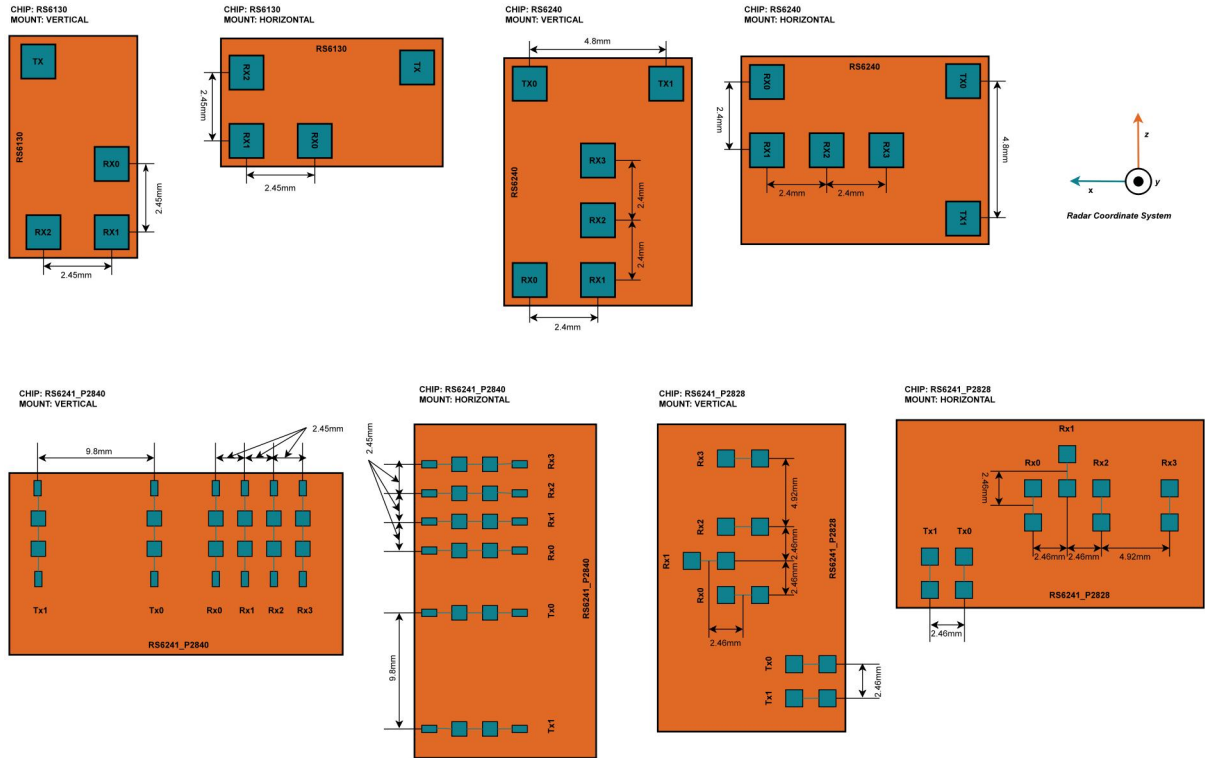


Figure 2-11 Radar Coordinate System Definition

The target coordinates in the radar coordinate system are the same for horizontal and vertical mounting. Using vertical mounting as an example, the target coordinates in the radar coordinate system are shown in Figure 2-12.

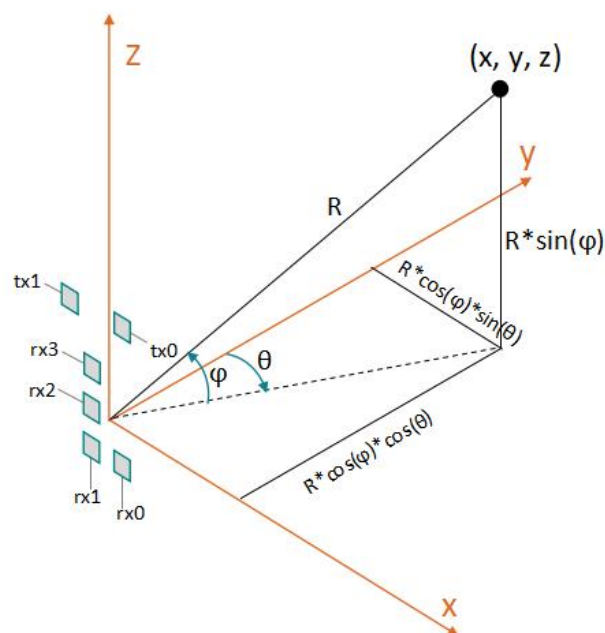


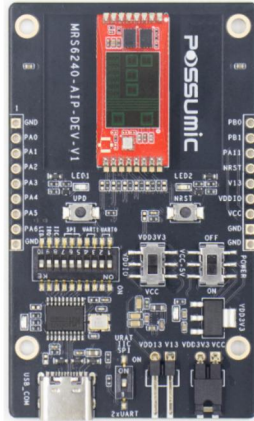
Figure 2-12 Positive Directions of the Radar Coordinate System

The two angles φ are the elevation θ angle and azimuth angle, and the arrows indicate their positive directions.

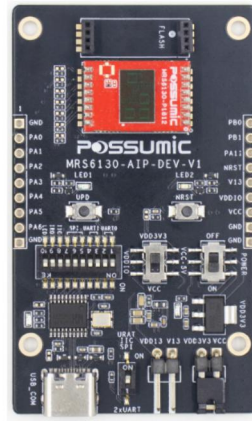
In the point cloud reference design, modify the default value of mount_type in the configuration interface *subsys/mmw/mmw_application/mmw_app_pointcloud_config.c* to configure the radar coordinate system used by the algorithm.

2.5.3.2 User Coordinate System

CHIP: RS6240
MODULE: MRS6240-P2512
DEV BOARD: MRS6240-AIP-DEV-V1



CHIP: RS6130
MODULE: MRS6130-P1812
DEV BOARD: MRS6130-AIP-DEV-V1



CHIP: RS6130
MODULE: MRS6130-P1806
DEV BOARD: MRS6130-AIP-DEV-V1



CHIP: RS6241
MODULE: MRS6241-P2828-M62
DEV BOARD: MRSx241-DEV-V1



CHIP: RS6241
MODULE: MRS6241-P2840-M81
DEV BOARD: MRSx241-DEV-V1

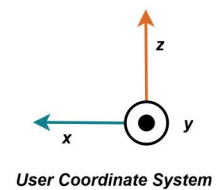
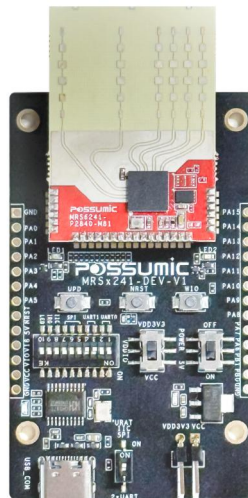


Figure 2-13 Development Board in the Side-Mounted Vertical Orientation

To provide a unified output coordinate system for different hardware assemblies, target coordinates generally must be converted from the radar coordinate system to the user coordinate system. In the point cloud reference design, target coordinates are converted before point cloud output so that all development boards use the same user coordinate system.

The conversion method for the user coordinate system depends on the radar coordinate system. The point cloud reference design defines the default radar coordinate system as vertical. The user coordinate system supports only the side-mounted vertical board orientation shown in Figure 2-13, with positive directions defined in Figure 2-14.

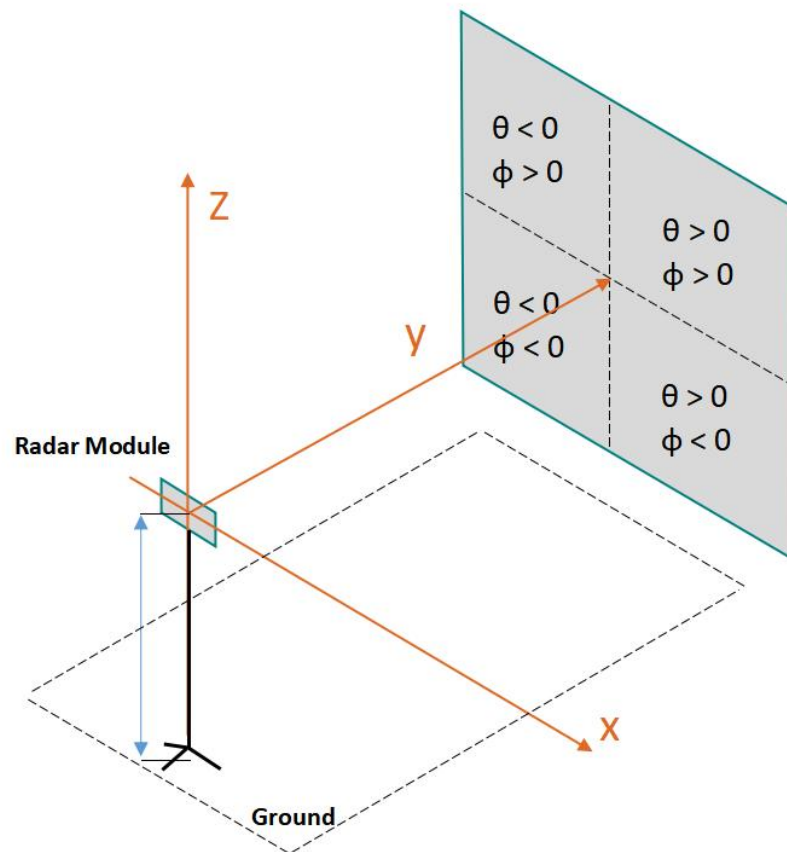


Figure 2-14 Positive Directions of the User Coordinate System

The radar coordinate system is configured as vertical because horizontal-angle accuracy is generally the primary concern. More antennas are therefore used to measure the horizontal angle and obtain higher angle-estimation accuracy.

2.5.4 RS6x_7x Angle Calibration

For RS6x_7x devices, software angle calibration is mandatory because antenna feed lines have unequal lengths. Without calibration, angle information is unusable. Angle calibration removes amplitude and phase errors introduced by antenna feed lines and receive channels so that the array signals satisfy the mathematical constraints. RS613x series devices read angle-calibration data from Flash and write it to registers.

RS6x_7x antenna calibration multiplies each receive channel by its corresponding calibration value. This method is not optimal, but it imposes the least stringent requirements on mass-production sites and equipment. For details about calculating calibration values, refer to the RS6xxx Production Test Guide.

The point cloud reference design includes a predefined set of calibration parameters, *g_rx_ant_calib_data*, obtained by averaging calibration values from multiple devices. During initial evaluation and development, these parameters can be used as the defaults.

For RS613x devices, angle-calibration parameters are read from Flash and written to registers when *mmw_ctrl_open()* is called. Therefore, the IQ data stored in the DataCube has already been calibrated.

For RS624x and RS7x series devices, *mmw_ctrl_open()* does not load calibration parameters from Flash into the calibration registers. After target IQ data is obtained, angle calibration must therefore be performed by calling *mmw_process_ant_data_calibrate_f32()*, as follows:

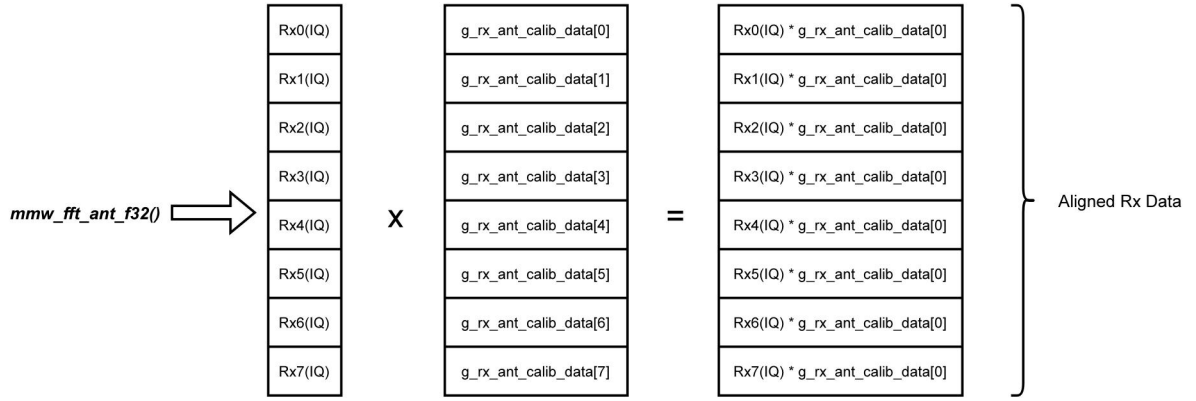


Figure 2-15 Phase Calibration Calculation Method

During phase calibration, *mmw_fft_ant_f32()* obtains the receive-channel IQ data at the specified range and Doppler indices. Each channel is multiplied by its corresponding calibration parameter in *g_rx_ant_calib_data* to produce the calibrated channel data *ant_aligned*, which is used for subsequent steps such as angle measurement.

2.5.5 RS6x_7x Angle Measurement Interface

The point cloud reference design assumes the side-mounted vertical orientation shown in Figure 2-13. For the RS6130, Rx2 and Rx1 are used for azimuth measurement, and Rx0 and Rx1 are used for elevation measurement. For the RS6240, Rx0/Rx1/Rx4/Rx5 are used for azimuth measurement. Two antenna groups can be used for elevation measurement: Rx3/Rx2/Rx1 and Rx7/Rx6/Rx5. The point cloud reference design uses only Rx3/Rx2/Rx1 for elevation measurement.

The following figure shows the implementation flow of the angle measurement interface *mmw_process_angle_process()*.

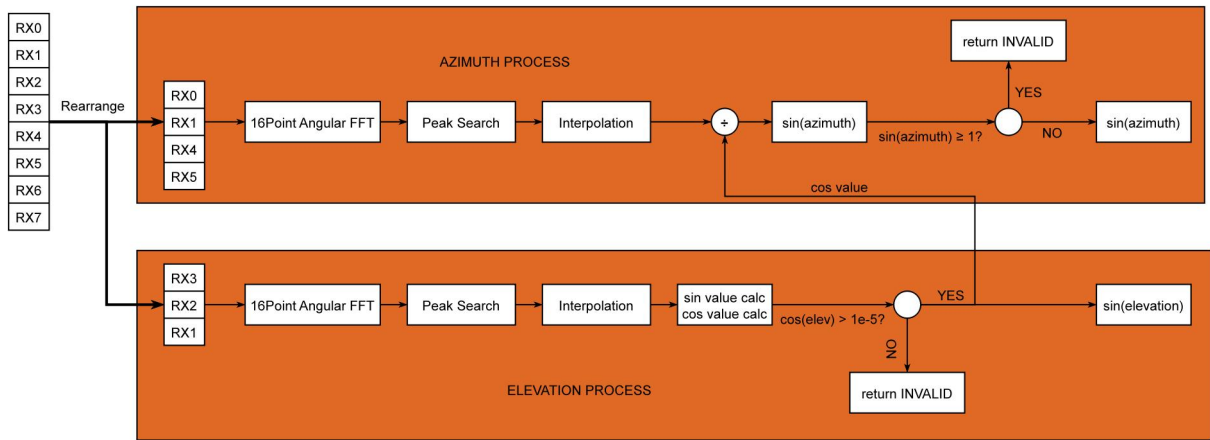


Figure 2-16 Angle Measurement Flow

The detailed description of the flow is as follows:

1. According to 2.5.2 The definition of an antenna describes the initiation of an antenna drop structure: *g_azi_geometry_struct* and *g_elev_geometry_struct*. Members of the Structure *ptr_geometry* records the antennae that need to be measured at levels and angles in inclination.
2. Arrange and select the receive-channel IQ data required for azimuth and elevation measurement. This data is obtained through *mmw_fft_ant_f32()*;
3. Pass *g_azi_geometry_struct* and *g_elev_geometry_struct* to *mmw_process_angle_process()*. The sequence recorded in *g_azi_geometry_struct.ptr_geometry* is used to measure the horizontal angle of arrival, which is not the azimuth angle defined above. The sequence recorded in *g_elev_geometry_struct.ptr_geometry* is used to measure the elevation angle;
4. Call *mmw_process_angle_calculate()*. According to the method in Section 2.5.1, use the elevation antennas to measure the elevation angle and the horizontal ϕ antennas to measure the sine of the horizontal angle of θ' arrival. Note that $\sin \theta'$ the latter is not the θ' defined azimuth angle at this stage;
5. Calculate range R from the target index in the DataCube. The Cartesian and polar coordinates shown in Figure 2-12 are calculated (x, y, z) (r, θ, ϕ) as follows:

$$\text{Polar Coordinates: } \begin{cases} r = R \\ \theta = \arcsin(\sin \theta' / \cos \varphi) \\ \varphi = \varphi \end{cases}$$

$$\text{Cartesian Coordinates: } \begin{cases} x = r \times \sin \theta' = r \times \cos \varphi \sin \theta \\ z = r \times \sin \varphi \\ y = \sqrt{r^2 - x^2 - z^2} = r \sqrt{1 - \sin^2 \theta' - \sin^2 \varphi} = r \times \cos \varphi \cos \theta \end{cases}$$

6. The preceding equations show that calculation of the y coordinate in Cartesian coordinates is subject to a constraint. The point cloud reference design applies this constraint by evaluating:

$$1 - \sin^2 \theta' - \sin^2 \varphi < \varepsilon$$

When the preceding condition is met, the y-coordinate calculation is close to singular, and the algorithm directly excludes these targets.

3 Presence Point Cloud Digital Signal Processing

The basic presence point cloud processing flow is the same as for the moving point cloud. The primary difference is that DataCubes from multiple frames are decimated to form a new presence DataCube.

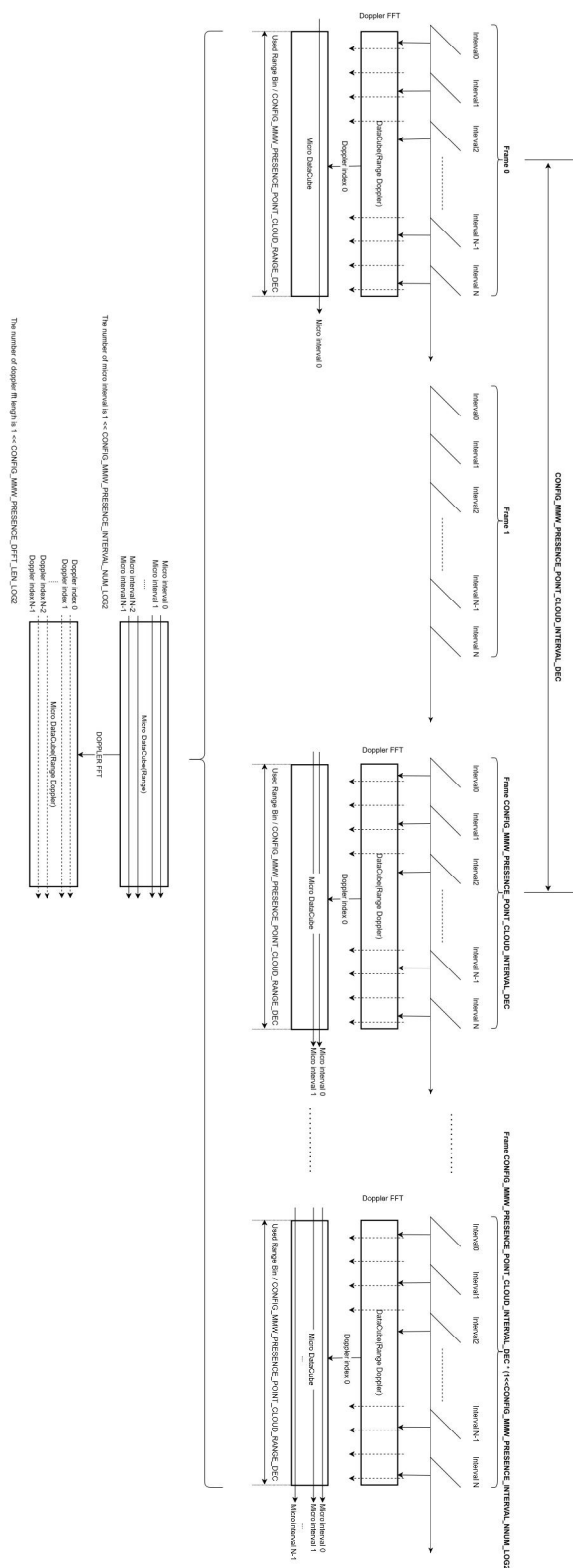


Figure 3-1 Presence Point Cloud DataCube Processing Flow

As shown above, every **CONFIG_MMW_PRESENCE_POINT_CLOUD_INTERVAL_DEC** frames, the presence point cloud processing module samples all range bins at one Doppler index as one sample in the slow-time dimension of the presence point cloud.

After the specified number of presence point cloud Intervals has been sampled in the slow-time dimension, a presence DataCube is calculated. The presence DataCube uses a FIFO structure. Therefore, after the maximum number of Intervals has been stored, a presence DataCube is calculated every **CONFIG_MMW_PRESENCE_POINT_CLOUD_INTERVAL_DEC** frames, followed by presence point cloud processing on the resulting Cube.

3.1 Detection Method Selection

When obtaining the presence point cloud target list, the SDK provides two detection methods: SNR threshold detection and CFAR detection. CFAR detection supports two modes: single-sided CFAR and double-sided CFAR.

Set the macro '**CONFIG_MMW_PRESENCE_POINT_CLOUD_PROCESS_METHOD**' to 0 (**MICRO_PROCESS_SNR_THRESHOLD_METHOD**) to select SNR threshold detection. This method determines a target by requiring the power of the cell under test, relative to the noise-floor power, to exceed a fixed threshold. It is suitable for relatively open environments, or for scenarios that require a high multi-target detection rate but have less stringent requirements for point cloud density and spurious points. Compared with CFAR detection, this method uses less heap memory.

Set the macro '**CONFIG_MMW_PRESENCE_POINT_CLOUD_PROCESS_METHOD**' to 1 (**MICRO_PROCESS_CFAR_METHOD**) and '**CONFIG_MMW_MICRO_POINT_CLOUD_CFAR_MODE**' to 0 (**MICRO_CFAR_MODE_SINGLE_SIDE**) to select single-sided CFAR. This mode estimates the background noise using only the noise reference cells ahead of the cell under test. It is suitable for simple, single-target (or few-target) environments with moderate requirements for detection rate, point cloud density, and spurious-point suppression.

Set the macro '**CONFIG_MMW_PRESENCE_POINT_CLOUD_PROCESS_METHOD**' to 1 (**MICRO_PROCESS_CFAR_METHOD**) and '**CONFIG_MMW_MICRO_POINT_CLOUD_CFAR_MODE**' to 1 (**MICRO_CFAR_MODE_BOTH_SIDE**) to select double-sided CFAR. This mode estimates the background noise using noise reference cells both ahead of and behind the cell under test. In complex environments, it provides a more reliable noise estimate from multiple noise cells and balances the background noise level. It is suitable when the area around the target is complex, clutter may be present ahead of the target, or high multi-target detection performance and spurious-point suppression are required. Double-sided CFAR requires an additional 10 KB of heap memory.

If a strongly reflecting object is located behind the target in the actual application, single-sided CFAR is recommended.

CFAR detection uses linear-threshold filtering so that different SNR thresholds are applied at different ranges. The SNR threshold is a straight line expressed as $SNR_{TH} = k * Range + b$, where Range is the range in m; SNR_{TH} is the SNR threshold in dB at that range; k is the slope; and b is the intercept. Configure the macro '**CONFIG_MMW_PRESENCE_POINT_CLOUD_CFAR_TH_DB**' to set the minimum SNR threshold (default: 8.5), as follows:

$$SNR_{TH_{Min}} = CONFIG_MMW_PRESENCE_POINT_CLOUD_CFAR_TH_DB$$

$$\begin{cases} SNR_{TH} = k * Range + b, SNR_{TH} > SNR_{TH_{Min}} \\ SNR_{TH} = SNR_{TH_{Min}}, SNR_{TH} \leq SNR_{TH_{Min}} \end{cases}$$

Note: The default detection method is double-sided CFAR. If SNR threshold detection is selected, setting '**CONFIG_MMW_PRESENCE_POINT_CLOUD_CFAR_TH_DB**' to 10.5 is recommended.

3.2 Angle Measurement

The angle measurement method for the presence point cloud is the same as for the moving point cloud. Refer to Section 2.5.

4 Configuring the Point Cloud Processing Subsystem

The point cloud processing subsystem provides default macros for all supported configuration items. Developers can override these macros in their projects to modify the default values.

4.1 Baseband Processing Configuration

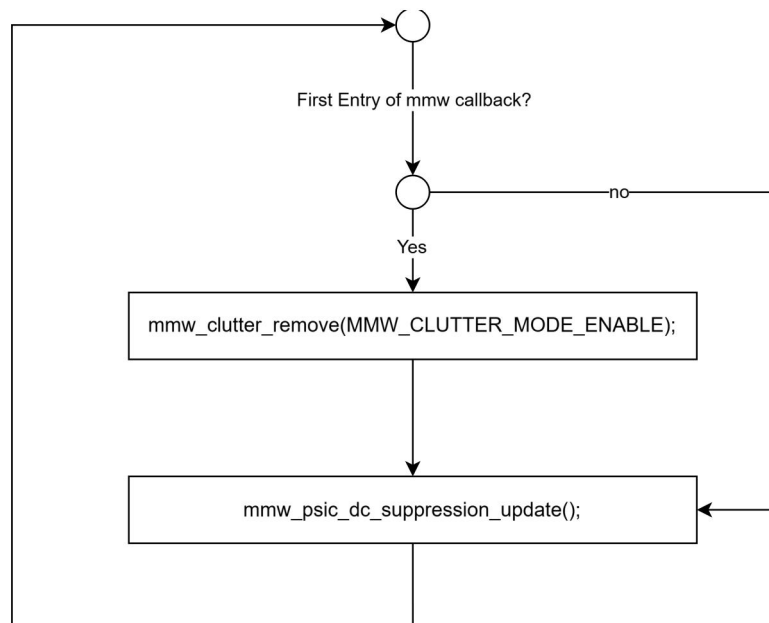
4.1.1 Static Clutter Removal Method

Use `CONFIG_MMW_POINT_CLOUD_BB_CLUTTER_RM` to configure the default static-clutter-removal function of the point cloud processing subsystem.

The configurable parameters are:

- `MMW_CLUTTER_REMOVAL_NONE`: Disable the hardware static-clutter filtering described in Figure 2-1;
- `MMW_CLUTTER_REMOVAL_DC`: Enable the hardware static-clutter filtering described in Figure 2-1. Software estimates and updates clutter caused by direct coupling, and hardware performs the removal;
- `MMW_CLUTTER_REMOVAL_ALL`: Enable the hardware static-clutter filtering described in Figure 2-1. Hardware automatically calculates and removes static clutter across the full range.

When `CONFIG_MMW_POINT_CLOUD_BB_CLUTTER_RM` is configured as `MMW_CLUTTER_REMOVAL_DC`, the application must implement direct-coupling update logic in the radar callback:



Function call sequence in mmw callback when enable `MMW_CLUTTER_REMOVAL_DC`

Figure 4-1 Radar Callback Scheduling Sequence When Configured as `MMW_CLUTTER_REMOVAL_DC`

The static clutter removal methods have the following restrictions:

1. If the presence point cloud is required or the radar is mounted on a moving platform, configure `CONFIG_MMW_POINT_CLOUD_BB_CLUTTER_RM` as `MMW_CLUTTER_REMOVAL_NONE`;
2. If stationary targets do not need to be detected and the presence point cloud is not required, configure `CONFIG_MMW_POINT_CLOUD_BB_CLUTTER_RM` as `MMW_CLUTTER_REMOVAL_ALL`;
3. If only the effect of Tx-Rx direct coupling on target detection within four range bins must be removed, configure `CONFIG_MMW_POINT_CLOUD_BB_CLUTTER_RM` as `MMW_CLUTTER_REMOVAL_DC`.

4.1.2 Static Clutter Convergence Time

Use `CONFIG_MMW_POINT_CLOUD_BB_CLUTTER_COVER_TIME` to configure the static-clutter convergence time. This parameter is passed directly to `mmw_clutter_factor_cfg()` and is effective only when the static clutter removal method is `MMW_CLUTTER_REMOVAL_ALL`. Larger values produce slower convergence. For details about its operating principle, refer to the '[RS6x 7x mmWave Radar Reference Manual.pdf](#)'.

Unless otherwise required, configure this value as 4 to balance clutter-estimation characteristics and convergence speed.

4.1.3 Receive-Antenna Gain and Transmit-Antenna Attenuation

- Use `CONFIG_MMW_POINT_CLOUD_BB_REC_P_GAIN` to configure receive gain. This parameter is passed to `mmw_rx_ana_gain_cfg()` and configures the same receive gain for all receive antennas. Unless otherwise required, retain the default value;
- Use `CONFIG_MMW_POINT_CLOUD_BB_TX_ATTENUA` to configure transmit attenuation. This parameter is passed to `mmw_tx_rf_att_set()`. Unless otherwise required, retain the default value. It is typically adjusted in applications where transmit power must be limited to meet regulatory requirements.

4.1.4 IF Filter Bandwidth

Use `CONFIG_MMW_POINT_CLOUD_BB_HPF_BW` to configure the IF filter bandwidth. For short detection ranges, the SNR of close targets is no longer a limiting factor. In this case, configure `CONFIG_MMW_POINT_CLOUD_BB_HPF_BW` as `HPF_BW_2M` to better suppress energy coupled from long range into short-range signals.

4.2 Moving Point Cloud Function Configuration

4.2.1 Maximum CFAR Target List Size Configuration

Use `CONFIG_MMW_MAX_MOTION_CFAR_NUM` to configure the maximum number of points read from the CFAR list. If the computational load is excessive or SRAM is insufficient, reduce this value to increase processing speed. If the actual number of CFAR points exceeds this value, the point cloud processing subsystem uses the first `CONFIG_MMW_MAX_MOTION_CFAR_NUM` points for subsequent calculations.

4.2.2 CFAR Threshold

Use `CONFIG_MMW_MOTION_CFAR_TH_LIN` to configure the CFAR threshold. This threshold is the linear multiplier by which the CFAR detection threshold on the power spectrum exceeds the noise-floor power. If the desired detection threshold is 12 dB above the noise-floor power, configure `CONFIG_MMW_MOTION_CFAR_TH_LIN` as $\text{power}(10, 12/10) = 15.8489$. The corresponding CFAR threshold is generally configured within 10 to 25 dB and must not exceed 30 dB.

4.2.3 Peak Grouping Configuration

Use `CONFIG_MMW_MOTION_PEAKGROUP_METHOD` to configure the peak-grouping method. For computational efficiency, peak grouping is generally configured as `PEAKGROUPING_DISABLE`. If different point-cloud densities are required for valid and invalid targets, try the available peak-grouping methods and determine whether they meet project requirements.

4.2.4 Point Cloud Filtering Algorithm Configuration

Configure the threshold for filtering noise points in the `mmw_psc_2nd_pass_filter()` point cloud filtering algorithm via the macro '`CONFIG_MMW_MOTION_2ND_PASS_THRES`'. The larger this value, the weaker the filtering effect.

`CONFIG_MMW_MOTION_2ND_PASS_THRES` must be greater than 0. The recommended value is 3.

4.2.5 Static Target Removal

Use `CONFIG_MMW_STATIC_RM_EN` to configure whether all targets with Doppler indices $[\text{dfft_len}/2-1, \text{dfft_len}/2, \text{dfft_len}/2+1]$ are filtered in CFAR, including software and hardware CFAR.

If the radar is stationary and relatively stationary targets do not need to be detected, enable this macro. Otherwise, stationary environmental targets rapidly fill the CFAR target list and waste processing resources.

When the radar is stationary, static target removal and static clutter removal overlap slightly. Static target removal directly removes relatively stationary targets in application logic, whereas static clutter removal learns and removes clutter at the digital signal processing layer.

Enabling static clutter removal allows temporarily stationary relative targets to be observed, such as a pedestrian crossing laterally with zero radial velocity. This is more broadly applicable but may produce a small number of false detections at zero velocity.

Customers can determine whether to enable static target removal according to project requirements. When static target removal is enabled and the radar is mounted on a stationary platform, static clutter removal should also be enabled to prevent energy leakage from very strong reflectors from extending beyond the static-target-removal range.

4.2.6 Low-Speed Filtering

Use `CONFIG_MMW_LOW_SPEED_FILTER_EN` to configure whether low-speed filtering is enabled.

If the 'static target removal' function filters too many stationary targets, use the low-speed filtering function to narrow the speed range for filtering stationary targets, allowing more subtle moving targets to not be filtered while filtering most of the interference in the environment.

Use `CONFIG_MMW_LOW_SPEED_FILTER_FACTOR` to configure the low-speed filtering range. This parameter determines the range as follows:

`[dfft_len/2-CONFIG_MMW_LOW_SPEED_FILTER_FACTOR,dfft_len/2+CONFIG_MMW_LOW_SPEED_FILTER_FACTOR]`

The valid range of `CONFIG_MMW_LOW_SPEED_FILTER_FACTOR` is 0 to 0.5.

4.3 Presence Point Cloud Function Configuration

4.3.1 Maximum Range Supported by the Presence Point Cloud

Use `CONFIG_MMW_PRESENCE_RANGE_BIN_NUM` to configure the maximum range bin searched and stored for the presence point cloud. The actual number of range bins used is $\text{Min}(\text{range FFT length}, \text{CONFIG_MMW_PRESENCE_RANGE_BIN_NUM})$. If the range FFT length exceeds `CONFIG_MMW_PRESENCE_RANGE_BIN_NUM`, determine whether this macro must be increased to match the range FFT length. Increasing this value increases the SRAM used by the presence point cloud.

4.3.2 Presence Point Cloud Detection Threshold

Use the macro '`CONFIG_MMW_PRESENCE_POINT_CLOUD_CFAR_TH_DB`' to configure the presence point cloud detection threshold (dB). A higher threshold reduces the detection capability for weak targets but lowers the false-alarm rate. For CFAR detection, this value controls only the minimum linear threshold.

The overall threshold offset can also be adjusted using the macro '`CONFIG_MMW_PRESENCE_POINT_CLOUD_CFAR_LINEAR_TH_OFFSET_DB`'. For SNR threshold detection, it has the same effect as '`CONFIG_MMW_PRESENCE_POINT_CLOUD_CFAR_TH_DB`'. For CFAR detection, it raises the entire linear threshold.

SNR threshold detection and CFAR detection currently share the preceding macros for threshold configuration. The recommended range for '`CONFIG_MMW_PRESENCE_POINT_CLOUD_CFAR_TH_DB`' is 8 to 15 dB, and the recommended range for '`CONFIG_MMW_PRESENCE_POINT_CLOUD_CFAR_LINEAR_TH_OFFSET_DB`' is -3 to 3 dB.

4.3.3 Number of Presence Point Cloud Intervals (log2)

Use `CONFIG_MMW_PRESENCE_INTERVAL_NUM_LOG2` to configure the number of Intervals used to calculate the presence micro-Doppler DataCube. The actual number is $1 < \text{CONFIG_MMW_PRESENCE_INTERVAL_NUM_LOG2}$.

The frame span of the presence point cloud DataCube is:

`CONFIG_MMW_PRESENCE_POINT_CLOUD_INTERVAL_DEC * 1 < CONFIG_MMW_PRESENCE_INTERVAL_NUM_LOG2`

Using more Intervals improves detection of distant presence points and points with slower variation, but significantly increases SRAM usage and slightly increases processing time.

This value must be less than or equal to `CONFIG_MMW_PRESENCE_DFFT_LEN_LOG2`. Refer to Chapter 3 for more information about presence point cloud processing and decimation.

4.3.4 Presence Point Cloud Doppler FFT Length (log2)

Use `CONFIG_MMW_PRESENCE_DFFT_LEN_LOG2` to configure the micro-Doppler FFT length for the presence point cloud. It must be greater than or equal to the number of presence point cloud Intervals configured by `CONFIG_MMW_PRESENCE_INTERVAL_NUM_LOG2`.

A larger micro-Doppler FFT length does not improve target detection, but it can increase point cloud density and processing time. Only values 4 and 5 are supported. Do not modify this value unless necessary.

4.3.5 Presence Point Cloud Range Downsampling Ratio

Use `CONFIG_MMW_PRESENCE_POINT_CLOUD_RANGE_DEC` to configure the range downsampling ratio for the presence point cloud. The point cloud processing submodule decimates and stores range data at this ratio. A larger ratio significantly reduces storage and computational load but decreases presence point cloud density.

For example, if `CONFIG_MMW_PRESENCE_RANGE_BIN_NUM` is 256 and `CONFIG_MMW_PRESENCE_POINT_CLOUD_RANGE_DEC` is 2, the range bins used to calculate the micro-Doppler DataCube are 0/2/4/... for a total of 127 bins. Other range bins are neither cached nor calculated.

Keeping the default value of 2 is recommended. If storage and computation pressure remains high, use a larger range downsampling factor. Refer to Section 3 for more details about presence point cloud downsampling.

4.3.6 Presence Point Cloud Interval Downsampling Ratio

Use `CONFIG_MMW_PRESENCE_POINT_CLOUD_INTERVAL_DEC` to configure the Interval downsampling ratio for the presence point cloud. Every `CONFIG_MMW_PRESENCE_POINT_CLOUD_INTERVAL_DEC` frames, the point cloud processing submodule obtains the range FFT data at the zero-velocity bin of the DataCube as one Interval sample for the micro-Doppler DataCube.

The frame span of the presence DataCube is:

`CONFIG_MMW_PRESENCE_POINT_CLOUD_INTERVAL_DEC * 1 <= CONFIG_MMW_PRESENCE_INTERVAL_NUM_LOG2`

To observe weaker presence points, configure a larger frame span for the presence DataCube.

4.4 Global Configuration

4.4.1 Chip Mounting Orientation

Use `CONFIG_MMW_CHIP_MOUNT_TYPE` to configure the radar coordinate-system orientation. Refer to Figure 2-11 for detailed definitions. The default is the coordinate-system orientation corresponding to a vertically mounted development board.

4.4.2 Adaptive Gain

Use `CONFIG_MMW_FFT_AUTOGAIN_EN` to enable adaptive gain for the hardware FFT. Do not enable adaptive gain in the following cases:

1. When reporting a DataCube, enabling hardware FFT adaptive gain requires reporting an additional gain coefficient, which is not supported by the current protocol;
2. When `mmw_point_cloud_process_sw_cfar()` is called to use software CFAR, the software CFAR does not support adaptive-gain restoration.

If only moving or presence point clouds are used, adaptive gain is recommended for a higher detection dynamic range. Adaptive gain is disabled by default.

For more information about adaptive gain, refer to the ['RS6x 7x mmWave Radar Reference Manual.pdf'](#).

4.4.3 Point Cloud Range

Use `CONFIG_MMW_RANGE_INDEX_THRESHOLD_MIN` and `CONFIG_MMW_RANGE_INDEX_THRESHOLD_MAX` to configure the point cloud calculation range. Only points with range indices within `[CONFIG_MMW_RANGE_INDEX_THRESHOLD_MIN, CONFIG_MMW_RANGE_INDEX_THRESHOLD_MAX]` undergo angle measurement and are output.

If you want to reduce the computational load for point clouds at uninteresting positions, use these macros to limit the range of distance indices.

4.4.4 Point Cloud Velocity Range

Use `CONFIG_MMW_POSITIVE_SPEED_INDEX_THRESHOLD_MIN` and `CONFIG_MMW_POSITIVE_SPEED_INDEX_THRESHOLD_MAX`, together with `CONFIG_MMW_NEGATIVE_SPEED_INDEX_THRESHOLD_MIN` and `CONFIG_MMW_NEGATIVE_SPEED_INDEX_THRESHOLD_MAX`, to configure the valid point-cloud velocity range.

For a target with positive velocity, angle measurement and output are performed only when its velocity-index offset from zero velocity is within `[CONFIG_MMW_POSITIVE_SPEED_INDEX_THRESHOLD_MIN, CONFIG_MMW_POSITIVE_SPEED_INDEX_THRESHOLD_MAX]`.

For a target with negative velocity, angle measurement and output are performed only when the absolute value of its velocity-index offset from zero velocity is within `[CONFIG_MMW_NEGATIVE_SPEED_INDEX_THRESHOLD_MIN, CONFIG_MMW_NEGATIVE_SPEED_INDEX_THRESHOLD_MAX]`.

5 Execution Time and Memory Usage

The point cloud reference-design test case mmw/mmw_application/test/mmw_point_cloud_ut.c demonstrates how to obtain module execution times through the SDK. The execution times of the modules are shown below:

Table 5-1 Execution Time per Call of Key Functions

Function Name	Execution Time
mmw_process_low_speed_filter	1us
mmw_process_1st_pass_filter	<1us
mmw_psic_noise_get_f32	4us
mmw_process_fft_ant_f32	9us
mmw_process_2nd_pass_filter()	1us
mmw_process_ant_data_calibrate_f32()	<1us
mmw_psic_2nd_pass_filter()	15us
mmw_process_angle_process()	15us

Table 5-2 Presence Point Cloud Execution Time per Frame

Module	Presence Point Cloud Detection Method	Execution Time
MRS6240_P2512	SNR Threshold	17 ms/frame
MRS6240_P2512	Single-Sided CFAR	13 ms/frame
MRS6240_P2512	Double-Sided CFAR	13 ms/frame
MRS6130_P1812	SNR Threshold	7.8 ms/frame
MRS6130_P1812	Single-Sided CFAR	18 ms/frame
MRS6130_P1812	Double-Sided CFAR	18 ms/frame

Configure the predefined macro CONFIG_DEBUG_SIZE_LEVEL=2 in the mmw_application Sample project to obtain SRAM usage. The code size of the point cloud processing subsystem in SRAM is 14.2 KB. Refer to the RS6x_7x mmWave Radar Development Quick Start Guide.

mmw	17.18K / 17596	745	21.45K / 21960
mmw_algorithm	8.31K / 8505	112	
cfar_casogo.c			
mmw_alg_ant_calibration.c	2.49K / 2548	32	
mmw_alg_debug.c			
mmw_alg_doa.c		116	36
mmw_alg_micro_cube.c	1.86K / 1904	44	
mmw_alg_pointcloud.c	3.84K / 3937		
mmw_application	5.90K / 6041	266	
mmw_app_micro_pointcloud.c	2.76K / 2828	56	
mmw_app_pointcloud.c	3.08K / 3159	64	
mmw_app_pointcloud_config.c		54	146
test			
mmw_point_cloud_ut.c			
mmw_point_cloud_vv.c			

Figure 5-1 SRAM Usage of the Point Cloud Subsystem

6 [Q & A]

Q: Why are all DataCube values sometimes read as 0?

A: It may be that the BaseBand is accessing the DataCube at this time, causing the CPU to read a value of all zeros from the DataCube.

Q: Why does the presence point cloud SNR threshold detection method sometimes output too few points?

A: When SNR threshold detection is used, lower the threshold as appropriate if the point density is insufficient. Alternatively, disable stationary-bin detection in the Doppler dimension during presence point cloud initialization (set the second bit of `cfar_stationary` to 0). This also increases the number of spurious points.

Disclaimer and Copyright Notice

The information provided by our company in this document is accurate to the best of our knowledge, but we do not guarantee that the content is completely error-free. All liability for any damages arising from the use of this document (including but not limited to indirect, incidental, or special losses) or any improper use resulting from non-compliance with the instructions herein (including but not limited to over-voltage, over-frequency, or over-temperature operations) is explicitly disclaimed.

The content in this document, including URL references, may differ from the actual content due to product version upgrades or other reasons, and is subject to change without notice.

THIS DOCUMENT IS PROVIDED "AS IS" WITH NO WARRANTIES. NO WARRANTY IS PROVIDED TO THIS DOCUMENT FOR ITS MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, NOR DOES ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

All liability, including liability for infringement of any patent rights, relating to use of information in this document is disclaimed. No licenses express or implied, by estoppel or otherwise, to any intellectual property rights are granted herein.

Copyright © 2026 Possumic Technology Co., Ltd. All rights reserved.